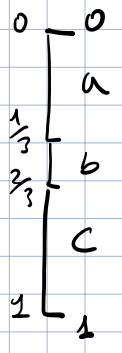
Dynamic Prefix Sums

Fenwick Tree (Binary Indexed tree, BIT)



a a ..... a b

0 0 ..... 0 1

Problem

Given $A[1,n]$, we would like to support

- $Sum(i) = \sum_{k=1}^{i} A[k]$

- $Add(i,v) : A[i] += v$

(Range $Sum(i,j) = Sum(j) - Sum(i-1)$)

Trivial Solutions

- Store A
  - $Sum(i) = \sum_{k=1}^{i} A[k]$ $\Theta(n)$ time
  - $Add(i, v) \rightarrow A[i] += v$ $O(1)$ time

- Precompute Prefix Sums P of A
  - $Sum(i) \rightarrow P[i]$ $O(1)$ time
  - $Add(i, v)$ increase any position $j \geq i$ in P by v $\Theta(n)$ time

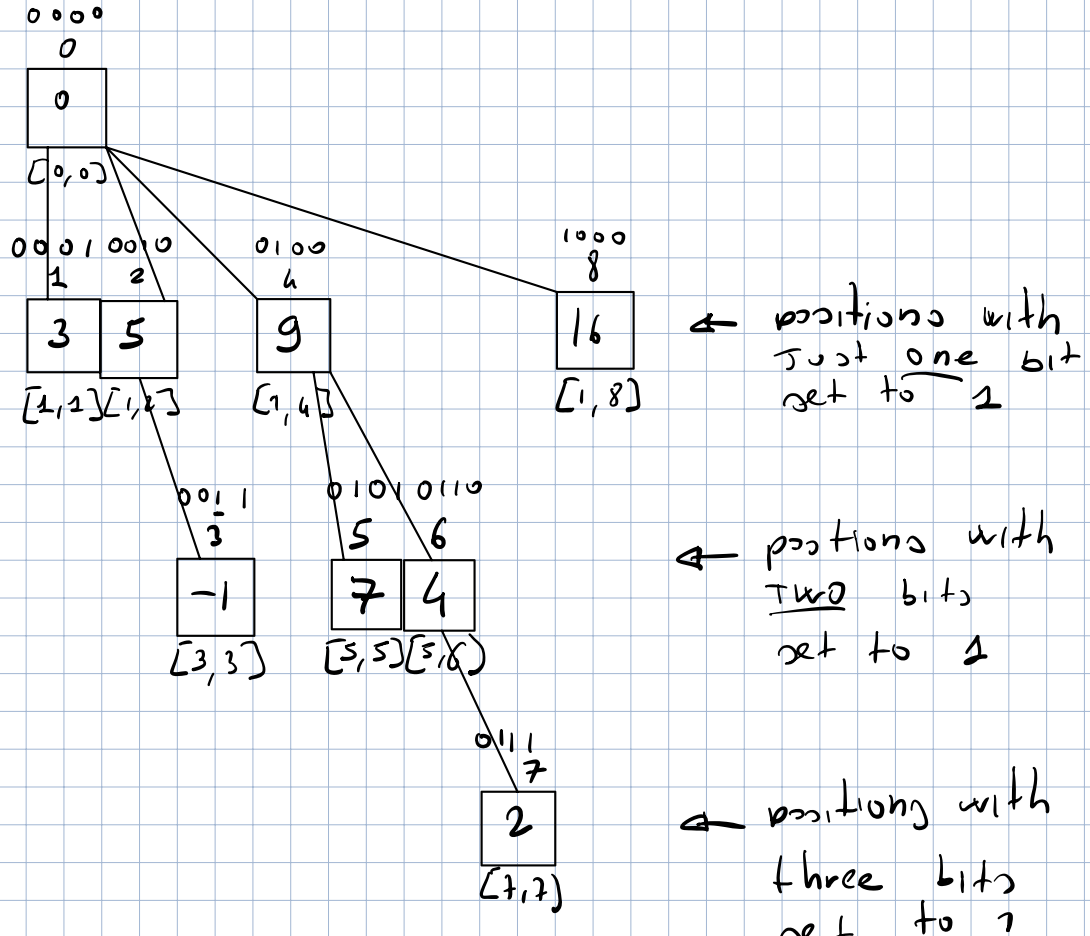
BIT supports both in $\Theta(\log n)$ time

	1	2	3	4	5	6	7	8
A	3	2	-1	5	7	-3	2	1

Relaxed version of the problem:

- Sum(i) $\sum_{k=1}^{i} A[k]$ but only for positions i which are a power of 2
- Add(i, v) $A[i] += v$

	1	2	3	4	5	6	7	8
A	3	2	-1	5	7	-3	2	1



FT	1	2	3	4	5	6	7	8
	3	5	-1	9	7	4	2	16

Sum(7) = 2+4+9 = 15

		value
7	0111	2
6	0110	4
4	0100	9
0	0000	0

parent(i) = i - (i & -i)

7 - (7 & -7)

0111 - (0111 & 1001) = 0110 = 6

0001

$i + (-i) = 2^N$

i $b_1\ b_2\ b_3\ b_4 \ldots\ b_4\ 1\ 0\ 0\ 0\ 0\ 0$

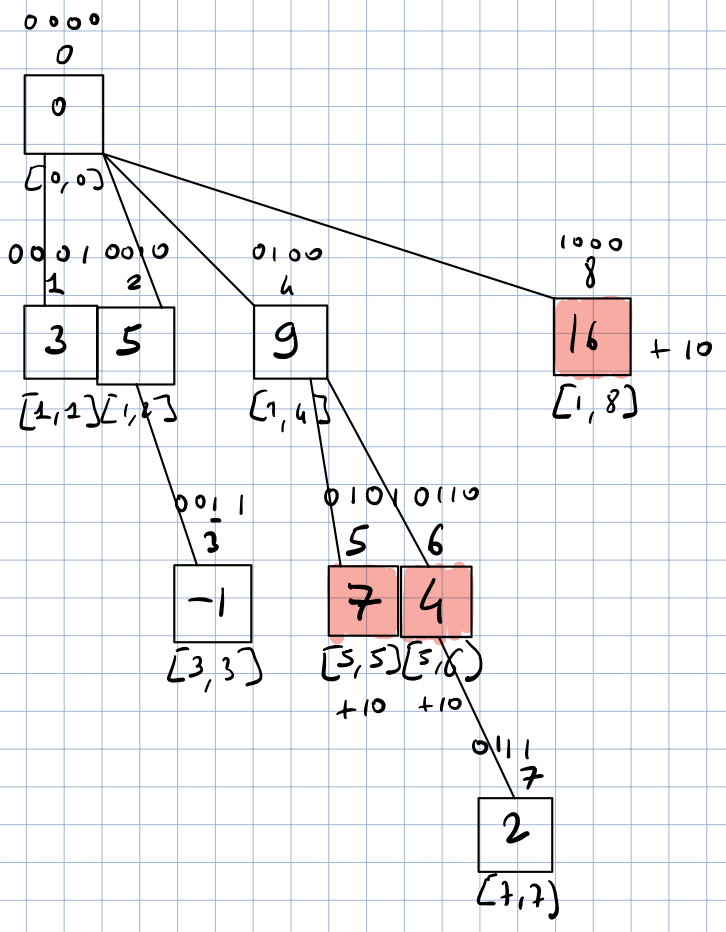
-i $\bar{b}_1\ \bar{b}_2\ \bar{b}_3\ \bar{b}_4 \ldots\ \bar{b}_4\ 1\ 0\ 0\ 0\ 0$

Sum(i)

```
s = 0
while i != 0
    s += FT[i]
    i = parent(i)  // i -= i & -i
return s
```

Complexity : $\Theta(\log n)$ time

Add(5, 10)



5	0101	+0001
6	0110	+0010
8	1000	+1000
16	10000	

move_right(i) = i + (i & -i)

parent(i) = i - (i & -i)

Add(i, v)

while i ≤ n

A[i] += v

i += i & -i

Complexity :

Counting Inversions

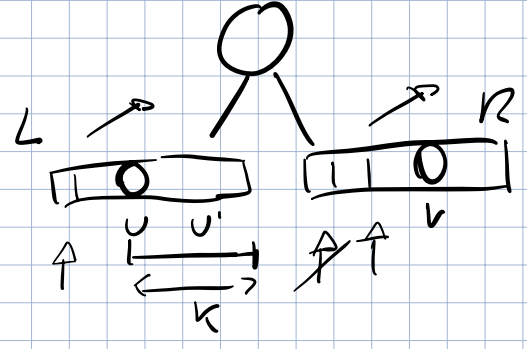
Given $A[1,n]$ of positive numbers $\leq n$,
Count the number of inversions in A

$i < j$ such that $A[i] > A[j]$

A 3 2 1 6 5 4 # inversions = 6

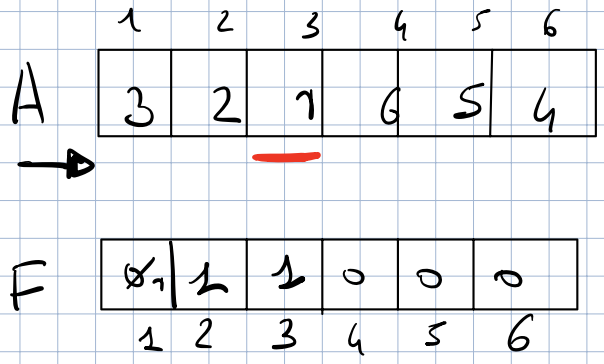
Solution

- Running Insertion Sort can count number of steps but $\Theta(n^2)$ time
- Merge Sort-Based solution in $\Theta(n \log n)$ time



if $V < U$, #-inversions = K

- Solution based on Fenwick Tree



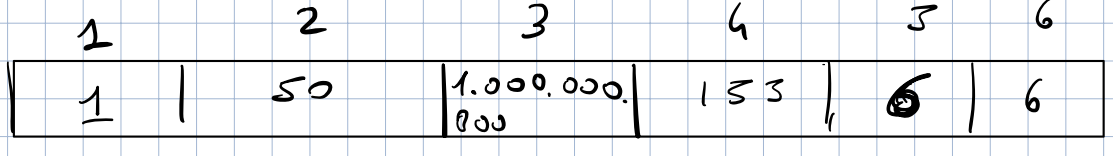
$i < 3$ such that $A[i] > A[3] = 1$

Do Range Sum $(1+1, 6) = 2$

$= 3 - \text{Sum}(3)$

- $\Theta(n \log n)$ time

If max value is too big, remap

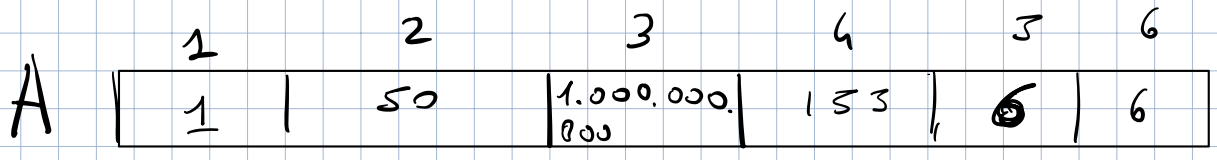


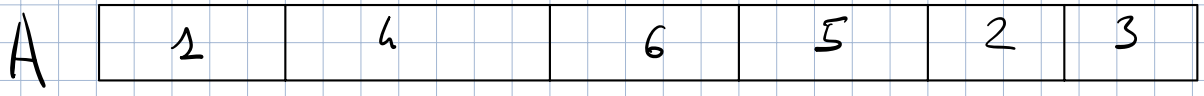
REMAPPING

- Sort in increasing order $\Theta(n \log n)$ time
- remap a value with its rank in

the ordering

	pos	rank
1	(, 1)	1
6	(, 5)	2
6	(, 6)	3
50	(, 2)	4
155	(, 4)	5
1B	(, 3)	6





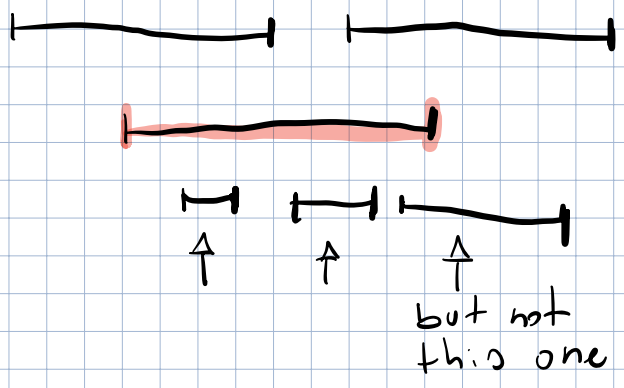
remap

Nested Segments

We are given segments

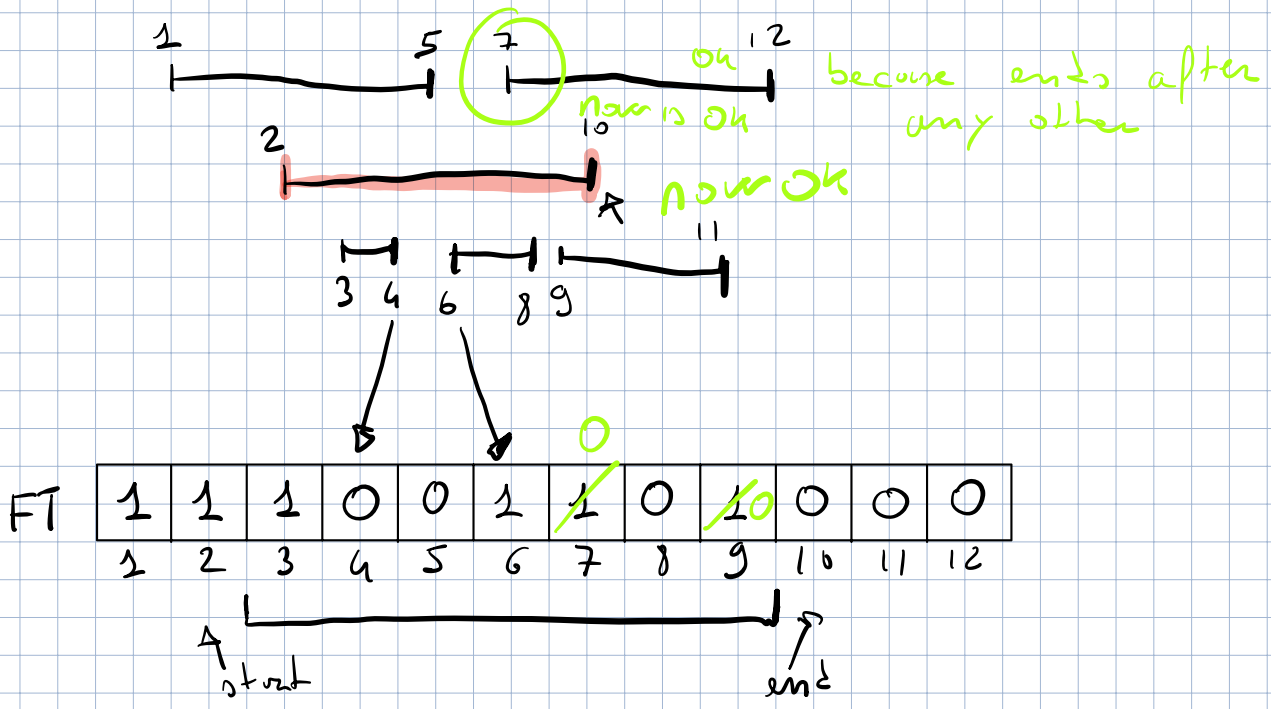
$$\langle l_1, r_1 \rangle, \langle l_2, r_2 \rangle, \ldots\ldots, \langle l_n, r_n \rangle$$

The goal is to report, for each segment, the number of segments it contains.



Solution in $\Theta(n \log n)$ time

- Remap the extremes if needed



RangeSum(3, 9) = 4 = number of segments starting within this segment

RangeSum(3, 9) = 2

Update the array

Given $A[?, n]$ initially $A[i] = 0 \ \forall i$

we would like to support:

- Access(i) = A[i]
- Update(i, j, v) $\forall k \in [i, j)$ $A[k] += v$

Example

	0	1	2	3	4	5	6	7	8
A	~~0~~	~~0~~	~~0~~	~~0~~	~~0~~	~~0~~	~~0~~	0	0
	5	5	~~10~~ 15	~~10~~ 15	~~10~~ 15	10	10		

Update(2, 6, 10)

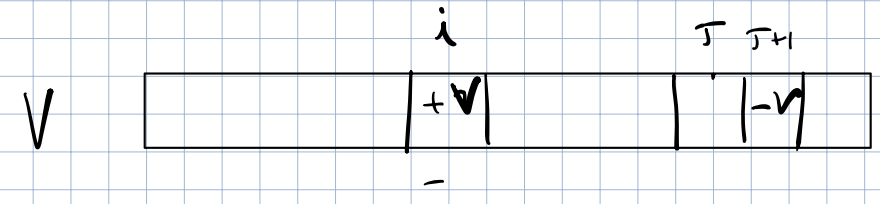
Update(0, 4, 5)

Solution: Both ops in $\Theta(\log n)$ time

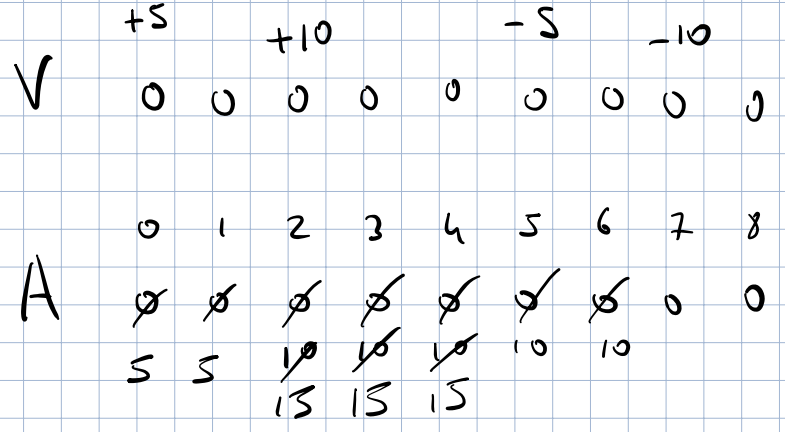
- Trivial keep A and Update(i, j, v) in $\Theta(j - i)$ time

Idea: use $V[1, n]$ such that

$$\forall i \quad A[i] = \sum_{k=1}^{i} V[k]$$



Update (i, j, v)



Update $(2, 6, 10)$ ←

Update $(0, 4, 5)$ ← $\begin{cases} Add_V(0, 5) \\ Add_V(5, -5) \end{cases}$

$Access_A(i) = Sum_V(i)$

both $\Theta(\log n)$ time with FT

Dynamic Prefix Sums with Range Update

Given $A[1, n]$

- RangeUpdate (i, j, v)
- Sum (i)

[- Access (i) = Sum (i) − Sum $(i-1)$]

	1	2	3	4	5	6	7	8	9	10
A	0	0	0	~~0~~ +10	~~0~~ +10	~~0~~ +10	0	0	0	0

RangeUpdate $(4, 6, 10)$

Sum $(6) = 30$

Solution runs $\Theta(\log n)$ time per query

Two Fenwick trees

FT, used in a similar way (as in the previous problem)

	1	2	3	4	5	6	7	8	9	10
V	0	0	0	0 +10	0	0	0 −10	0	0	0

RangeUpdate $(4, 6, 10)$

$Sum(i) = FT_1 . Sum(i) \cdot i$

RangeUpdate (ℓ, 6, 10) Ok if $\ell == 1$

1 2 3 4 5 6 7

+10 −10

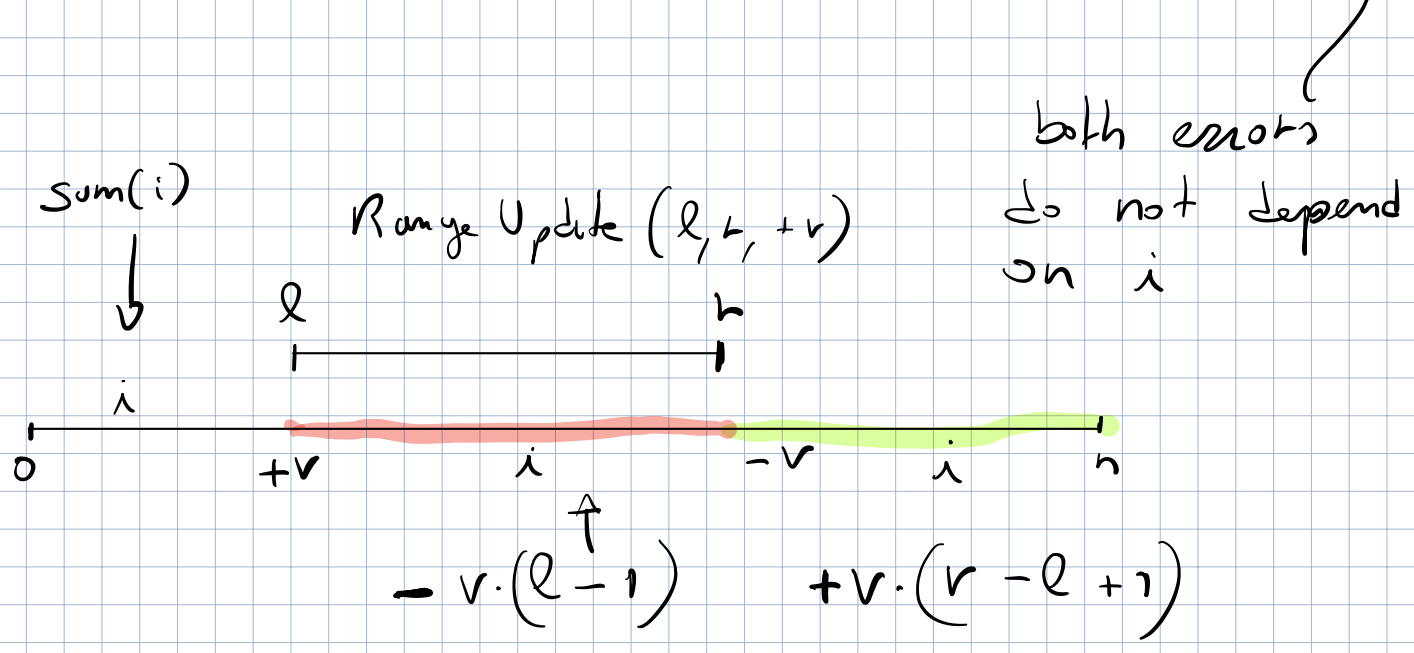
$Sum(6) = FT_1 . Sum(6) \cdot 6 = 60$

RangeUpdate (ℓ, r, $+v$)

- $+v$ at position ℓ
- $-v$ at position r
- $Sum(i) = FT_1 . Sum(i) \cdot i$

· What's wrong ?

		correct	ours
①	if $0 \le i < \ell$	0	$0 \cdot i = 0$
②	if $\ell \le i \le r$	$v \cdot (i - \ell + 1)$	$i \cdot v = v \cdot (\ell - 1) + v(i - \ell + 1)$
③	if $r < i \le n$	$v \cdot (r - \ell + 1)$	$0 \cdot i = -v \cdot (r - \ell + 1)$



Keep an array FT_2 of corrections with

Update the array solution

- FT_2.RangeUpdate(l, r, $-v(l-1)$)
- FT_2.RangeUpdate(r+1, n, $v\cdot(r-l+1)$)

$$Sum(i) = FT_1.Sum(i)\cdot i + FT_2.Sum(i)$$